



NOMBRES

Juan Pablo Rovayo Delgado

NIVEL

Quinto Semestre

ASIGNATURA

Desarrollo de Sistemas de Información

DOCENTE

Ing. José Luis Naranjo Villota

PERIODO ACADEMICO

Abril – Agosto

Proyecto

Manta360

Repositorio del proyecto

<https://github.com/Manta360/Manta360>

Despliegue

<https://manta360.vercel.app>

Tablero Equipo Jira

<https://manta360.atlassian.net/jira/software/projects/KAN/summary>

Reporte de Gestión e Historial Técnico

1. Resumen de tickets gestionados en Jira

Participación organizada por épicas en el proyecto KAN (Manta360):

Épica KAN-25 — Reportes, Quejas y Mantenimiento (Finalizado)

ID	Descripción	Estado
KAN-26	Creación de reportes de incidencias en contratos activos (backend + UI arrendatario)	Finalizado
KAN-27	Cambio de estado de quejas (Pendiente → En proceso → Resuelto) por el arrendador	Finalizado
KAN-28	Suite de testing de permisos cruzados y rendimiento del mapa (Vitest)	Finalizado
KAN-31	Inhabilitación / rehabilitación de arrendadores y propiedades (Municipio)	Finalizado

Épica KAN-50 — Cumplimiento Completo del Municipio y Búsqueda (Finalizado)

ID	Descripción	Estado
KAN-51	Completar gestión municipal: listado de usuarios (arrendadores/arrendatarios), filtros y panel Usuarios	Finalizado
KAN-52	Completar estadísticas obligatorias (zonas, incidencias con ceros, top 5 arrendadores)	Finalizado
KAN-53	Búsqueda completa en backend (location, precio, servicios) restringida a arrendatario autenticado	Finalizado

Épica KAN-59 — Calidad, Pruebas E2E y Cierre (En revisión)

ID	Descripción	Estado
KAN-60	Pruebas E2E del flujo principal hasta DISPONIBLE + sincronización contrato→propiedad	Finalizado
KAN-61	Suite de permisos y seguridad (12 escenarios Vitest)	Finalizado
KAN-62	Documentación y entorno reproducible (README, database/BDD.sql , docs/)	Finalizado

2. Aportes técnicos en código (commits y Pull Requests)

Autor Git: Juan Pablo (rovayojuanpablo@gmail.com) — ~33 commits propios (más merges de integración al main).

Pull Requests autorados

PR	Título / alcance	Estado
#8	KAN-25: incidencias, inhabilitación municipal y suite de testing	Merged
#22	KAN-50: municipio, usuarios, estadísticas y búsqueda	Merged
#23	KAN-59: E2E, seguridad y documentación de cierre	Merged

Evidencia de commits representativos

Commit / tema	Aporte técnico
feat: [KAN-26] --- incidencias	API POST /api/incident-reports + UI de reporte
feat: [KAN-27] --- quejas	Transiciones de estado y sección “Quejas y mantenimiento”
feat: [KAN-31] --- inhabilitación	Endpoints admin + bloqueo de login con motivo municipal
feat: [KAN-28] --- testing	Vitest: permisos cruzados 403/400/200 y mapa (<2s)
KAN-51 / KAN-52 / KAN-53	Backend admin users/stats + filtros catálogo + UI municipio
fix/test: [KAN-60]	finalizeContractAndSynchronizeProperty + script E2E con ROLLBACK
test: [KAN-61]	src/tests/security-permissions.test.ts (12 escenarios)
docs: [KAN-62]	README rúbrica, BDD.sql , DER, manual, despliegue, TESTING.md

Capas tocadas: backend (API Routes Next.js + pg), frontend (paneles por rol), base de datos (script SQL / seed demo) y calidad (Vitest + E2E de integración).

Propuestas y Aprendizaje

1. Propuesta estratégica A — Suite automatizada de seguridad y permisos (KAN-28 → KAN-61)

Problema: En un monolito modular con tres roles (Municipio, Arrendador, Arrendatario), las regresiones de autorización (401/403/404) y la fuga de datos sensibles (passwordHash) son difíciles de detectar solo con pruebas manuales.

Propuesta aportada: Establecer una capa de pruebas automatizadas con Vitest que ejecute rutas reales de la API bajo identidades cruzadas: visitante sin sesión, acceso a recursos ajenos, registro ilegal como MUNICIPIO, renovación fuera de plazo, propiedad ocupada/inhabilitada, arrendador inhabilitado, contratos duplicados y no serialización de hashes. La evolución de KAN-28 a KAN-61 consolidó 12 escenarios en security-permissions.test.ts, ejecutables con npm test / Vitest, reduciendo el riesgo de “permisos por confianza” en el cierre.

2. Propuesta estratégica B — Cierre reproducible: E2E del flujo + documentación canónica (KAN-60 / KAN-62)

Problema: Había dos cuellos de botella de cierre: (1) inconsistencia al terminar un contrato (la propiedad no siempre volvía a DISPONIBLE); (2) la rúbrica exige entorno reproducible (esquema BDD, DER, README, demo), no solo código disperso.

Propuesta aportada

Unificar la sincronización contrato → propiedad en finalizeContractAndSynchronizeProperty y validarla con un script E2E (npm run db:test-e2e-happy-path) que recorre el flujo principal bajo BEGIN/ROLLBACK y PG_TEST_*, sin ensuciar datos.

Entregar un paquete documental único: database/BDD.sql (14 tablas + seed), docs/ (arquitectura, modelo de datos, despliegue, manual por rol, pruebas/demo) y README alineado a la rúbrica, con tablero Jira enlazado.

3. Conclusión reflexiva

El trabajo en Manta360 reforzó competencias de ingeniería full-stack (Next.js 15, PostgreSQL vía pg, paneles por rol) y de calidad (pruebas de seguridad, E2E, documentación). El mayor reto técnico fue integrar módulos (contratos, propiedades, municipio, incidencias) sin romper invariantes de estado ni exponer secretos. En el plano grupal, Jira + GitHub (PRs #8, #22, #23) obligaron a comunicar alcance, criterios de aceptación y revisión cruzada; el cuello de botella no fue “escribir código”, sino alinear comportamiento de negocio y evidencia verificable para la evaluación. Como aprendizaje crítico: la arquitectura limpia y las pruebas no son un “extra” de cierre, sino la condición para que un sistema multi-rol sea defendible ante un evaluador y ante el propio equipo.